SL2DT-07: User Governance & Access

> **Series:** SL2DT — Sumo Logic to Dynatrace | **Notebook:** 7 of 11 | **Created:** April 2026 | **Last Updated:** 10/14/2026

Overview

Goal of this step: translate Sumo's RBAC model (roles + search filters + capabilities) into Dynatrace Platform IAM (groups + policies + bucket scoping). Do not defer this work — IAM decisions are made in Wave 1 of the migration, not at the end.

Governance drift — users without proper policies, buckets without proper scoping, SSO not mapped — causes cutover delays more often than query translation issues. Get this right before Wave 2.

Table of Contents

1. What You'll Produce
2. Sumo → Dynatrace IAM Mapping Model
3. Bucket-Scoped Policies — Implementation
4. SSO & User Provisioning
5. Role-Based Dashboard/Monitor Access
6. Audit Trail Parity
7. Governance Pitfalls to Avoid
8. Step Exit Criteria
9. References

Prerequisites

Requirement	Details
Audience	Platform admins + security team
Inputs	`inventory/roles.json`, `users.json`, `rbac-mapping.md` from SL2DT-02
Dynatrace access — Platform Token	For querying audit events (`fetch dt.system.events`): `storage:system:read` and `storage:buckets:read` on the `dt_system_events` bucket. Policy and group definitions are read through the Account Management API with the OAuth client below
Dynatrace access — OAuth client	**The Terraform resources used in this notebook's HCL examples (`dynatrace_iam_group`, `dynatrace_iam_policy`, `dynatrace_iam_policy_bindings_v2`) are OAuth-client-only** — a Platform Token cannot drive them. Provision an OAuth client (`DT_CLIENT_ID` / `DT_CLIENT_SECRET` / `DT_ACCOUNT_ID`) with scopes `account-idm-read`, `account-idm-write`, `iam-policies-management`, `account-env-read`
SSO context	Existing IdP details (Okta, Entra, Ping)
Prior reading	IAM-01 (Platform IAM fundamentals), IAM-07 (bucket policies)

1. What You'll Produce

Artifact	Purpose
`iam/groups.tf`	Terraform for IAM groups (one per Sumo role)
`iam/policies.tf`	Policies (scoped by bucket)
`iam/bindings.tf`	Group→policy bindings
`iam/sso-mapping.md`	IdP claim → group membership rule
`iam-rollout-plan.md`	Sequence: shadow-mode → cut-over → retire-legacy

2. Sumo → Dynatrace IAM Mapping Model

The Three Layers

```
Sumo Role              Dynatrace IAM Model
═══════════════        ═══════════════════════
Role name       →      Group name
Capabilities    →      Policy permissions
Search filter   →      Policy bucket/scope filter
User assignment →      Group membership (often via SSO claim)
```

Example Mapping

Sumo role: `prod-readers`
- Search filter: `_sourceCategory=prod/*`
- Capabilities: Dashboard view, Search, Monitor view (no edit)
- Assigned users: 82

Dynatrace equivalent:

```
# Group
resource "dynatrace_iam_group" "g_prod_readers" {
  name = "g_prod_readers"
  description = "Read-only access to production logs + dashboards"
}

# Policy (bucket-scoped)
resource "dynatrace_iam_policy" "p_prod_readers" {
  name = "p_prod_readers"
  environment = var.tenant_uuid
  statement_query = &lt;&lt;-EOT
    ALLOW storage:buckets:read WHERE storage:bucket-name = "custom_logs_prod";
    ALLOW storage:logs:read, storage:events:read, storage:metrics:read
    WHERE storage:bucket-name = "custom_logs_prod";
    ALLOW document:documents:read;
  EOT
}

# Binding
resource "dynatrace_iam_policy_bindings_v2" "b_prod_readers" {
  group = dynatrace_iam_group.g_prod_readers.id
  environment = var.tenant_uuid
  policy = [dynatrace_iam_policy.p_prod_readers.id]
}
```

> **Critical — `dynatrace_iam_policy_bindings_v2` re-assigns all policies bound to a group on every apply.** Per the `dynatrace_iam_policy_bindings_v2` resource docs (Dynatrace provider docs) — also mirrored, JavaScript-free, at the provider repo (Dynatrace GitHub): *"This resource re-assigns all policies bound to a group, so every policy that should remain bound must be specified in the configuration; otherwise, it will be unbound."* The single-policy example above is illustrative — if `g_prod_readers` already has other policies bound (in Dynatrace itself or in a separate Terraform resource), applying this block with only `p_prod_readers` listed will **unbind those other policies**. List every policy that should remain bound to the group in the same `policy = [...]` array.

Policy Language

Dynatrace policies use a declarative grant syntax:

```
ALLOW &lt;permission&gt;, &lt;permission&gt;, ...
  WHERE &lt;condition&gt;;
```

Every read of bucket data needs **two** grants: the table permission (`storage:logs:read`, `storage:events:read`, …) **and** `storage:buckets:read` — *"Grants permission to read records from Grail buckets. Required additionally to a table permission."* A policy with only the table permission reads nothing.

Common bucket-scope conditions:

- `storage:bucket-name = "bucket_x"`
- `storage:bucket-name IN ("a", "b", "c")`
- `storage:bucket-name startsWith "custom_logs_prod"`
- `storage:table-name = "logs"` (on `storage:buckets:read` — every bucket of one table)

Conditions take `=`, `!=`, `IN`, `NOT IN`, `startsWith`, `NOT startsWith` and `MATCH`; there is no `LIKE`.

> **Sources:** IAM policy statements (DT docs).

3. Bucket-Scoped Policies — Implementation

Bucket scoping is the primary isolation mechanism. Apply it for every team/environment boundary.

Policy Patterns

Pattern 1 — Read-only to one bucket

```
ALLOW storage:buckets:read
  WHERE storage:bucket-name = "custom_logs_prod";
ALLOW storage:logs:read, storage:events:read
  WHERE storage:bucket-name = "custom_logs_prod";
```

Pattern 2 — Read all, write via a deliberate unscoped grant

```
ALLOW storage:buckets:read
  WHERE storage:table-name = "logs";
ALLOW storage:logs:read;
// Storage writes cannot be condition-scoped (live-verified 07/2026) —
// grant unscoped, only to the team's ingest service user, and control
// where data lands via OpenPipeline routing, not IAM conditions.
ALLOW storage:logs:write;
```

Pattern 3 — Admin on multiple buckets

```
ALLOW storage:buckets:read
  WHERE storage:bucket-name startsWith "custom_logs_prod";
ALLOW storage:logs:read, storage:events:read
  WHERE storage:bucket-name startsWith "custom_logs_prod";
ALLOW storage:logs:write;
ALLOW document:documents:read, document:documents:write;
ALLOW settings:objects:read, settings:objects:write
  WHERE settings:schemaId = "builtin:monitoring.slo";
```

> **Storage writes cannot be bucket-scoped (live-verified 07/2026):** `storage:logs:write WHERE storage:bucket-name ...` is rejected with `Invalid condition name`. Grant writes unscoped in a deliberately-assigned policy and route data via OpenPipeline. Wildcard permissions (`storage:*:read` , `ALLOW *`) are also rejected — enumerate.

Pattern 4 — Full admin (limit to platform engineering)

Do not author a wildcard policy (`ALLOW *` is rejected by the API). Bind the built-in **Admin User** managed policy — or, for a classic-parity custom grant, combine `environment:roles:viewer` + `environment:roles:manage-settings` with enumerated storage/settings permissions.

Avoid

- Binding admin-level policies to team-level groups. Escalation risk.
- Broad enumerated grants where a scoped read would do — harder to audit.

- Read permissions without WHERE clauses — silently grants tenant-wide access. (Storage *writes* cannot take WHERE at all — grant them deliberately and sparingly.)

Validate Policies

After applying, verify a test user in the group can (and cannot) do what the policy says:

> **For teams spanning multiple source categories by component type** (e.g., a database team needing access to `_sourceCategory=*/db/*` regardless of application), a structured `comp:<component>/bu:<business-unit>/app:<application>` security context format enables a single `MATCH('comp:db*')` boundary to work across all applications. See **IAM-04: Policy Authoring** and **ORGNZ-06: Security Context** for the design pattern.

```
// Verify recent activity from the environment audit trail
//
// IAM policies, groups and bindings live in Account Management — they are not a Grail data object
// (`dt.iam.policies` fails with UNKNOWN_DATA_OBJECT). Read their definitions through the Account
// Management API, and their change history in the account audit log.
//
// Activity inside the environment IS queryable, as AUDIT_EVENT records in dt.system.events.
// Audit events carry user.id (not user.email).
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| summarize {calls = count(), users = countDistinctExact(user.id)}, by: {event.provider, event.type}
| sort calls desc
| limit 20
```

4. SSO & User Provisioning

SCIM / SAML Integration

Most migrations preserve the same IdP (Okta / Entra / Ping). Map IdP groups to Dynatrace groups via the SAML federation's group claim or SCIM group push.

SAML — group claim mapping. There is no fixed attribute name. In the federation's **Attribute mapping**, set **Federated** to the attribute your IdP sends group claims in (the docs' Entra ID example is the Microsoft `…/identity/claims/groups` claim URI; other IdPs send their own attribute name, for example `groups`). Then, on each Dynatrace group, add the **SAML Group Attribute Values** — the IdP group names or IDs — that map to it:

```
groups = "g_prod_readers,g_payments_team"     # example: whatever attribute
your IdP sends
```

On a federated user's sign-in, Dynatrace matches the incoming group claims against the SAML Group Attribute Values and adds the user to each matching group. A comma-separated value is split into separate groups, so send group names or IDs, not a full LDAP DN.

SCIM push (preferred for lifecycle management):
- The IdP's SCIM application provisions users and groups into Dynatrace (one-way, IdP → Dynatrace)
- Membership changes propagate with the IdP's SCIM synchronization
- A user disabled or removed in the IdP is disabled in Dynatrace; how quickly depends on the IdP's provisioning cycle — measure it during the pilot rather than assuming it is immediate

> **Sources:** SAML (DT docs) — *"you need to specify the SAML Group Attribute Value in the attribute mapping configuration that contains the groups/roles of a user from your IdP"*; SCIM (DT docs) — *"if a user is disabled or removed from the IdP, it will also disable the user in Dynatrace"*, *"SCIM synchronization is one-way from the IdP to Dynatrace."*

User Lifecycle

Event	Sumo Behavior	Dynatrace Behavior
New hire	Manual add in Sumo admin	SCIM push → auto-assigned to groups
Team change	Role manually reassigned	SCIM push reassigns group memberships
Termination	Manual disable	SCIM push removes all memberships

Provisioning Tracker

```
| User | Sumo Role | Target DT Group | SSO Push Confirmed |
|------|-----------|-----------------|--------------------|
| jane@example.com | prod-readers | g_prod_readers | ✅ 2026-05-01 |
| ... |
```

5. Role-Based Dashboard/Monitor Access

Dashboards, notebooks, and workflows have their own access model (via `document:documents:*` permissions) separate from bucket scoping.

Document Permission Levels

Permission	Effect
`document:documents:read`	Read documents of the document service
`document:documents:write`	Create and update documents
`document:documents:delete`	Delete documents
`document:documents:admin`	Admin permissions for documents (platform admins only)

The policy reference lists no "owner" or "shared with" conditions for these permissions, so IAM does not decide *which* documents a user sees — sharing does. A document is visible beyond its owner only when it is shared with a user, a group or the environment, which is why the share defaults below matter.

> **Sources:** IAM policy statements (DT docs).

Default Shares

Set dashboard defaults during migration so new dashboards are readable by the team:

- Every team dashboard: shared with the team's IAM group (read)
- Every team notebook (operational): shared with team (read)
- Every team workflow: owned by team admin group (read/write)

Content Folders

Dynatrace Documents have a flat namespace with tags/shares, not folder hierarchy like Sumo. Map Sumo folders to tags:

Sumo Folder	Dynatrace Tag
`Payments/Prod`	`team:payments`, `env:prod`
`Identity/Shared`	`team:identity`
`Archive`	`status:archived`

6. Audit Trail Parity

Sumo's audit log captures user actions in one place. Dynatrace splits them across two records:

- **Environment activity** — *"In Latest Dynatrace, audit events are stored in Grail as dt.system.events, queryable with Dynatrace Query Language (DQL)"*, filtered with `event.kind == "AUDIT_EVENT"`. *"Audit events are retained for one year in Grail, compared to 30 days in the classic API."*
- **Access changes** — groups, policies, SSO: *"The Account Management audit log (IAM changes, SSO configuration, budget changes) stays a separate system in both models."* It is read in Account Management, not with DQL, and *"Audit log data is stored for up to 10 years (3650 days)."*

Sumo audit categories → Dynatrace equivalents

Sumo	Dynatrace
Login	`event.kind == "AUDIT_EVENT"`, `event.type == "LOGIN"`
Monitor / settings created or updated	`event.provider == "SETTINGS"`, `event.type` `CREATE` / `UPDATE` / `DELETE`
Dashboard / notebook sharing and ownership	`event.provider == "DOCUMENTS"` (`ENV_SHARE_CREATE`, `ENV_SHARE_DELETE`, `DOCUMENT_TRANSFER_OWNER`, …)
API calls	`event.provider == "API_GATEWAY"` or `"CLASSIC_API"`, `event.type` = HTTP method

Sumo	Dynatrace
Search executed	`event.kind == "QUERY_EXECUTION_EVENT"` (also in `dt.system.events`; carries `user.email` and the query)
User / role changes	Account Management audit log (not in Grail)

The `event.provider` / `event.type` values above are the ones a test tenant recorded over seven days on 10/05/2026 — run the query below on yours to see the full set. **FAQ-24** §7 and **IAM-07** cover audit reporting in depth.

> **Sources:** Upgrade from classic audit logs (DT docs), Account Management audit logs (DT docs).

Query audit events

```
// Dynatrace environment audit events, by source and action
fetch dt.system.events, from:-24h
| filter event.kind == "AUDIT_EVENT"
| summarize c = count(), by:{event.provider, event.type}
| sort c desc
```

Retention

Environment audit events are stored in the system bucket `dt_system_events` and kept for one year (see above). The `audit_logs` bucket from SL2DT-03 holds audit-trail **log sources** you ingest, not Dynatrace's own audit events. If compliance needs environment activity beyond one year, plan an export. Account-level IAM changes are kept for up to ten years in the Account Management audit log.

Confirm the system bucket's retention on your tenant:

```
fetch dt.system.buckets
| filter name == "dt_system_events"
| fields name, retention_days
```

7. Governance Pitfalls to Avoid

Pitfall 1 — "We'll do IAM at the end"

Deferring IAM means rebuilding ingest (bucket decisions are coupled to policy scope). Do IAM in Wave 1.

Pitfall 2 — Overly broad initial groups

Tempting: start with one group `all-users` with `ALLOW *`. Migration pressure compounds: "we'll tighten later." Later never comes, and when it does, revoking access breaks people's workflows.

Fix: start with the Sumo role structure as-is (one DT group per Sumo role, same scope). Refactor later with data.

Pitfall 3 — Unscoped dashboard writes

A dashboard created by a team member ends up without share settings, then becomes unreachable when they leave. Every dashboard should have a group owner, not a user owner.

Fix: enforce via PR review during SL2DT-06. Check the default-share policy.

Pitfall 4 — SSO group-name mismatch

Okta pushes group name `payments-platform-prod`; Dynatrace group is `g_prod_readers`. Users log in but get no permissions.

Fix: document SSO-claim → DT-group mapping in `iam/sso-mapping.md`. Test with at least one user per group before cutover.

Pitfall 5 — Missing audit events for bucket-level access

Bucket read/write permissions are evaluated per-query, so "who queried what" is answered by query-execution events, not by the `AUDIT_EVENT` records above.

Fix: confirm `event.kind == "QUERY_EXECUTION_EVENT"` records are present in `dt.system.events` before cutover, and agree retention for both records with compliance (§6).

8. Step Exit Criteria

G7 — Governance Ready

- [ ] IAM groups created (one per Sumo role)
- [ ] Policies created with bucket scoping
- [ ] Group ↔ policy bindings applied
- [ ] SSO mapping documented and tested with at least one user per group
- [ ] Dashboard/notebook share defaults configured
- [ ] Environment audit events (`dt.system.events`, one year) and the Account Management audit log reviewed; export planned if compliance needs longer retention
- [ ] Rollout plan documented (shadow-mode → cutover → retire legacy)

Next step: SL2DT-08 — Automation & GitOps (Monaco/Terraform for config promotion, CI/CD integration).

---

9. References

Dynatrace IAM

- Identity and access management (DT docs)
- Manage user permissions and policies (DT docs)
- Access tokens (DT docs)
- Platform tokens (DT docs)

Sumo Logic users and roles (source)

- Sumo Logic users and roles (Sumo Logic docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace or Sumo Logic. Always verify information against the official Dynatrace documentation and Sumo Logic documentation.